Lab 7 Preprocessing Maps

2026-01-18

Maps

By now, you should feel comfortable creating basic interactive charts in python. This week, we'll step up the complexity and work with maps. Maps can be quite time consuming to produce.

Before you start plotting, think about your data:

What kind of location data do I have? (e.g., latitude/longitude, postcodes, addresses, shapefiles, GeoJSON)

What format does my chosen plotting package require?

Would a different package make this easier? (and what format does that one need?)

Imports

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(plotly)
```

```
## Loading required package: ggplot2

##
## Attaching package: 'plotly'

## The following object is masked from 'package:ggplot2':
##
##     last_plot

## The following object is masked from 'package:stats':
##
##     filter
```

```
## The following object is masked from 'package:graphics':
##
##     layout
```

```
library(ggplot2)
library(googlePolylines) #Introduced this week
```

```
## Warning: package 'googlePolylines' was built under R version 4.3.3
```

```
file_id <- "1ymbNqfv9s6YGZzN93HFKAhjg0Z5xZXV1"
url <- paste0("https://drive.google.com/uc?id=", file_id)

df <- read.csv(url)
```

1. Find data

Martins runs only data has a column called maps.

- What are the types of data contained? What other observations can you make? Consider how to decode this information. Print a single run. Notice the number of escapes \ appearing.

2. Create dataframe

Taking a single datapoint from Martins Runs can you create a dataframe of longditudes and latitudes.

> **HINT**:
>
> You will probably use google Polylines.
>
> Break into steps based on your answer to 1).
>
> The polyline may now include characters such as '\' gsub is a great method for cleaning ie `gsub("\\\\", "\\", poly, fixed = TRUE)`

```
##        lat      lon
## 1 51.43859 -3.17363
## 2 51.43856 -3.17366
## 3 51.43844 -3.17372
## 4 51.43835 -3.17374
## 5 51.43821 -3.17372
```

3. Last 20 Runs (Optional)

You can just work with the above dataframe if you prefer. The following is a challenge! If you do attempt it, ensure you understand what is going on.

Retrieve the most recent 20 runs and store them in a DataFrame similar to the previous one, but with an additional column for date. Each point should have a date, so the date will appear multiple times.

```
##        lat      lon       date
## 1 51.43859 -3.17363 21-07-2025
## 2 51.43856 -3.17366 21-07-2025
## 3 51.43844 -3.17372 21-07-2025
## 4 51.43835 -3.17374 21-07-2025
## 5 51.43821 -3.17372 21-07-2025
## 6 51.43816 -3.17372 21-07-2025
```

4. Save as csv

```r
write.csv(coords_df, "map.csv", row.names = FALSE)
```